

COMPENG 2DX3: Microprocessor Systems Project

Final Project Report – AC8874

Anushka Chauhan - Chauha30 – 400598874

Submitted on: 7 April, 2026

Instructors: Dr. Shahrukh Athar, Dr. Mohamed Elamien, Dr. Yaser M. Haddara

As a future member of the engineering profession, the student is responsible for performing the required work in an honest manner, without plagiarism and cheating. Submitting this work with my name and student number is a statement and understanding that this work is my own and adheres to the Academic Integrity Policy of McMaster University and the Code of Conduct of the Professional Engineers of Ontario. Submitted by [Anushka Chauhan, Chauha30,400598874]

Table of Contents

Design Overview	3
Features	3
General Description	4
Block Diagram	5
Device Characteristics Table	5
Detailed Description	6
Distance Measurement	6
Visualization	8
Application Note, Instructions, and Expected Output	8
Limitations	12
Circuit Schematic	13
Programming Logic Flowchart(s)	14
References	16

Design Overview

Features

Microcontroller (MSP432E401Y)

- **High-Performance Core:** Arm® Cortex®-M4F 32-bit CPU with a Floating-Point Unit (FPU).
- **System Bus Speed:** Operates at 22 MHz using the internal PLL for consistent timing.
- **Memory Architecture:** 1024 KB Flash for storing the program and 256 KB SRAM for scan data.
- **Operating Voltage:** Dual-rail power architecture utilizing 3.3V DC for logic/sensing and 5.0V DC for the motor drive.
- **Serial Communication Specs:** UART module configured for high-speed 115200 Baud Rate data transmission to the host PC.
- **I2C Interface:** Dedicated 100 kHz serial bus used for high-speed Time-of-Flight sensor data acquisition.
- **Hardware Value:** The complete development kit utilized for this product is valued at \$256.99 CAD.
- **GPIO:** Flexible pins used for the 2-button interface and 4-LED status display.

Distance Sensing (VL53L1X ToF)

- **Technology:** Uses a 940 nm invisible laser and a "Time-of-Flight" (ToF) sensor to find distances.
- **Range:** Can accurately measure objects up to 400 cm (4 meters) away.
- **Performance:** Works regardless of the object's colour or reflectance.
- **Hardware Reset:** Uses the XSHUT pin to properly initialize the sensor at startup.

Motor Driving (ULN2003 & Stepper)

- **Driver Chip:** Uses a Darlington Transistor Array to handle the high current needed by the motor.
- **Built-in Protection:** Includes clamp diodes so the motor doesn't damage the microcontroller.
- **Precision Movement:** Moves the sensor in small 11.25° steps (32 readings per full 360° circle).
- **Return-to-Home:** Automatically unwinds the motor after each scan to prevent wire tangles.

User Interaction & 3D Output

- **Control Buttons:** Use PJ0 to start a physical scan and PJ1 to send the data to your computer.
- **LED Status Display:** Onboard LEDs (D1-D4) flash to tell you if the system is ready, scanning, or sending data.
- **Programming Language (Receiver):** Raw data is processed on the host side using MATLAB for coordinate transformation and 3D reconstruction.
- **Visualization:** Generates an interactive 3D Point Cloud that allows users to rotate and analyze the scanned environment digitally.

General Description

The AC8874 acquires spatial data using a VL53L1X Time-of-Flight (ToF) sensor, which represents a significant advancement over standard infrared intensity sensors. The signal acquisition process begins when the sensor emits a 940 nm laser pulse; it then utilizes an internal SPAD (Single Photon Avalanche Diode) receiving array to detect the returning photons. The "Analog-to-Digital" conversion happens internally within the VL53L1X module, where the time-of-flight of the light is converted into a precise digital distance value (in millimetres). This digital data is then polled by the MSP432E401Y microcontroller via the I2C protocol at a clock speed of 100 kHz, ensuring that the distance measurement is independent of the target's colour or surface reflectance.

The communication between the hardware and the host computer is managed through a high-speed Universal Asynchronous Receiver/Transmitter (UART) interface. Once the user triggers the data transfer by pressing the PJ1 button, the microcontroller serializes the stored scan data (including Scan ID, Step ID, and Distance) into a standardized string format. This data is transmitted at a Baud Rate of 115200 bps over a USB-to-Serial bridge to the host PC. This method of communication allows for the reliable transfer of the 32 discrete data points collected during each 360-degree rotation, providing a seamless link between the embedded system and the processing software.

The final stage of the process utilizes a custom MATLAB processing suite to transform raw measurements into a visual model. The software calculates the spatial coordinates by applying trigonometric transformations to the raw distance (d) and the angular position (θ) of the 28BYJ-48 stepper motor, specifically using the formulas $y = d \cdot \cos(\theta)$ and $z = d \cdot \sin(\theta)$. To ensure the quality of the 3D point cloud, the script executes a data-cleaning algorithm that filters out measurement spikes and sensor noise by averaging values between neighbouring points. The resulting output is an interactive 3D structural model that connects adjacent samples to accurately render the geometry of the scanned environment.

Block Diagram

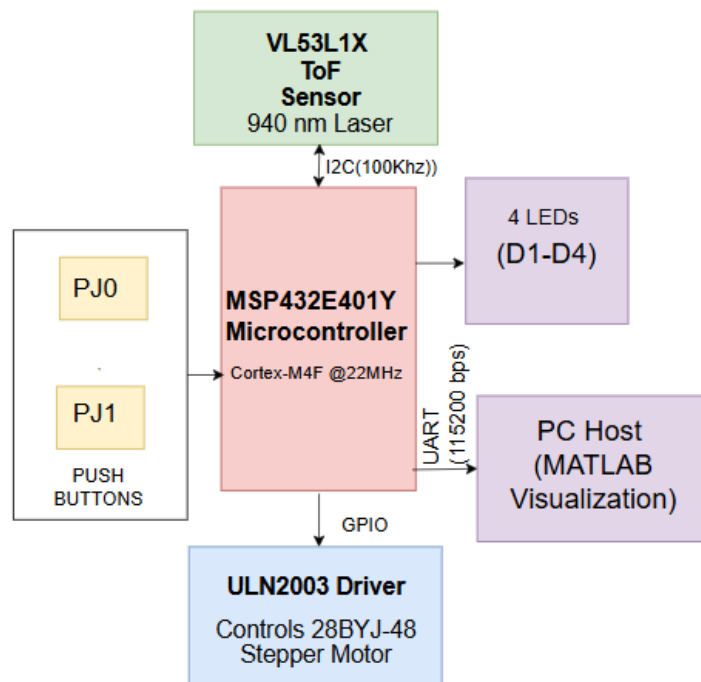


Figure 1: Block Diagram

Device Characteristics Table

Feature	Specification	Technical Details and Pin Mapping
Microcontroller Bus Speed	22 MHz	Established via PLL
Serial Communication	115200 bps	UART Protocol
I2C Bus Speed	100 kHz	MCU communication with ToF sensor
Distance Sensing Pin	I2C/GPIO	SDA(PN4), SCL(PN5), and XSHUT(PG0)
Motor Control Pins	GPIO Output	PL1, PL2, PL3, PL4 connected to ULN2003
User Input Pins	GPIO Input	PJ0(scan trigger) and PJ1(data transmission)
Visual Output Pins	GPIO Output	PN1, PN0, PF4, PF0 (status LEDs D1-D4)
Operation Voltage	3.3V/5.0V	3.3V for ToF Sensor 5.0V for Stepper Motor

Detailed Description

Distance Measurement

Acquisition

The AC8874 achieves high-precision spatial mapping by utilizing the interaction between two specialized optical components within the VL53L1X sensor: a 940 nm VCSEL (Vertical Cavity Surface Emitting Laser) emitter and a SPAD (Single Photon Avalanche Diode) receiving array. The VCSEL emitter projects a narrow, invisible infrared beam toward the target. Unlike standard LEDs, this laser emission is pulsed, allowing the system to distinguish its own signals from ambient light. The reflected photons are captured by the SPAD array, which is a highly sensitive semiconductor capable of triggering an electrical response when it detects a single photon. The sensor can operate with a minimum detection range of 40 mm and a maximum range of 4000 mm. By using a Time-to-Digital Converter (TDC), the sensor acts as a high-speed stopwatch, measuring the exact duration between the laser pulse emission and the photon return to calculate distance based on the constant speed of light. The internal processor of the VL53L1X converts the recorded time-of-flight into a digital distance value measured in millimetres (mm). This unit provides the fine resolution necessary for architectural mapping and obstacle detection. To ensure measurement stability, the MSP432E401Y is programmed to provide the sensor with a 50ms timing budget per sample. This duration allows the SPAD array to collect a sufficient number of photons to calculate a reliable average, effectively filtering out momentary optical noise in the raw signal.

The results of the distance acquisition are communicated to the microcontroller via the I2C (Inter-Integrated Circuit) serial protocol. The MSP432E401Y serves as the Controller, initiating data requests to the sensor over the SDA (Serial Data) and SCL (Serial Clock) lines (pins PN4 and PN5) at a bus speed of 100 kHz. The distance data is sent as 16-bit digital packets, ensuring measurement accuracy even in noisy environments. This digital interface outperforms analog alternatives by removing the need for the MCU's onboard ADC. To construct a 360-degree spatial slice, the acquisition process is digitally synchronized with the mechanical actuation of the stepper motor. The system is configured to take exactly 32 measurements per full rotation. After the motor completes an 11.25° step, the microcontroller pauses motion to allow the sensor to perform a static measurement. This discrete sampling method ensures that each of the 32 data points is geometrically aligned with a specific angular coordinate, providing a clean and organized dataset for the final MATLAB 3D visualization.

Data Processing, Formulas Used and Example

The conversion of raw spatial data into the Cartesian XYZ plane occurs post-acquisition within the MATLAB environment on the host PC. During the scanning phase, the MSP432E401Y records data in a Polar coordinate system, where each point is defined by a distance (d) in millimetres and an angular position (θ) determined by the motor's 11.25° step increments. To preserve processing power on the microcontroller, these raw values are transmitted via UART and processed only once they reach MATLAB.

The software then transforms these polar coordinates into a 3D grid using trigonometric identities. In this model, the X-axis represents the longitudinal displacement (the distance between sequential slices as the device is moved), while the Y and Z axes represent the horizontal and vertical coordinates of the circular scan. The formulas used for this

transformation are:

d = distance measured by ToF sensor
 θ = angular position of the stepper motor in radians
Y = Horizontal Displacement from the sensor center
Z = Vertical Displacement from the sensor center

$$Y = d \cdot \cos(\theta)$$

$$Z = d \cdot \sin(\theta)$$

For Example, suppose the motor angle is 45° and the sensor returns a distance of 250mm

First, the angle is converted to radians $\theta_{rad} = 45^\circ \cdot \frac{\pi}{180^\circ} = 0.785 \text{ rad}$

Calculating Y: $250 \text{ mm} \cdot \cos(0.785) = 176.847 \text{ mm}$

Calculating Z: $250 \text{ mm} \cdot \sin(0.785) = 176.706 \text{ mm}$

Calculating X: for the first slice, it will be $X = 0$, and as more scans are taken, it will increase X based on the distance specified in the code for physical displacement

By mapping every measurement (d, θ) at each longitudinal interval (X), MATLAB generates a full 3D Point Cloud. This data is preprocessed to filter out noise, such as readings below 40 mm and above 4000 mm, as well as unusual spikes, ensuring that the final visualization accurately reflects the room's structural geometry.

Implementation of Displacement

Displacement in the context of the AC8874 Spatial Mapper refers to the linear distance travelled by the entire prototype along the X-axis, which represents the depth of the environment being mapped. While the stepper motor and ToF sensor are responsible for capturing the circular cross-section of a room (the Y and Z coordinates), displacement provides the third dimension required to stack these circular slices into a continuous 3D map. Without tracking displacement, the software would simply overwrite the same 2D circle repeatedly, failing to reconstruct the longitudinal structure of the hallway. In this project, displacement was implemented manually to ensure spatial consistency. To maintain a stable and uniform height for every scan, the prototype was placed on a chair, which served as a mobile platform. This setup ensured that the sensor remained at the same level with the floor throughout the entire mapping process. The mapping followed a strict synchronization between physical movement and software logic: the device performed a full 360-degree rotation at a starting position ($X = 0 \text{ mm}$), after which the chair was manually moved forward along the hallway by exactly 250 mm.

To ensure this physical movement was accurately captured in the digital model, the displacement value was explicitly defined within the Keil C source code. In the Keil environment, a variable was created to represent the fixed 250 mm increment. For every new scan initiated by the user, the microcontroller increments a "Scan Counter." The code then calculates the current longitudinal position using the following logic:

$$X_pos = \text{Scan Counter} \times 250 \text{ mm}$$

This calculated X-coordinate is bundled with the distance and angle data and transmitted via UART to the PC. By performing this calculation on the microcontroller, the system ensures

that every data packet contains a complete 3D coordinate set before it even reaches the visualization software. By keeping this displacement constant at 250 mm within the code, the resulting 3D point cloud maintained a uniform scale. This method allowed the series of individual 2D circles to be stacked accurately, transforming them into a 3D view of the hallway.

Visualization

The final stage of the AC8874 system architecture is the digital reconstruction of the physical environment using MATLAB. This visualization software transforms the raw data received from the microcontroller into a geometrically accurate 3D point cloud. The procedure is managed by a MATLAB script that oversees data acquisition from the hardware, applies trigonometric transformations to the data, and performs data cleaning to ensure the final graph accurately represents the hallway.

To achieve 3D mapping, the system utilizes the native MATLAB Engine and its specialized communication and mathematical libraries. The primary functions used in the implementation include `serialport()` to initialize the link with the MSP432E401Y at the 115200 Baud Rate, and `readline()` to capture the incoming strings. The script then uses `sscanf()` to parse the comma-separated packets sent by the MCU, extracting the scan number, step index, raw distance, and the pre-calculated X-coordinate. For the spatial logic, `cos()` and `sin()` functions translate the polar distance into Y and Z Cartesian coordinates. The final visualization is produced using `plot3()`, which allows the software to place points in a 3D grid and draw lines between them to form a mesh. The procedure for producing the 3D map follows a structured four-step sequence within the MATLAB environment. First, the script enters a Data Acquisition loop, storing every incoming line until it receives an "END" signal. Second, the Coordinate Assignment phase uses the `linspace` function to evenly distribute 32 measurements around a 360° (2π radians) circle, while the X-axis value is pulled directly from the MCU's transmitted data, which is scaled to centimetres in MATLAB for plotting. Third, a Data Cleaning algorithm is applied to remove spikes or noise; any measurement under 40 mm, over 4000 mm, or showing a sudden jump compared to its neighbours is replaced by the average of the points around it. Finally, the 3D Rendering phase uses a loop to identify each unique scan, close the circular loop of 32 points, and layer these slices at the precise 25 cm intervals established in the Keil code to create a 3D view of the scanned environment.

Application Note, Instructions, and Expected Output

i. The Coordinate System (XYZ)

To accurately map a physical environment, the AC8874 translates sensor readings into a standard 3D Cartesian grid. The axes are defined as follows:

- **YZ-Plane (The Vertical Slice):** This plane represents the width (Y-axis) and height (Z-axis) of the hallway. These coordinates are derived from the 32 radial distance measurements taken as the ToF sensor rotates 360° .
- **X-Axis (The Longitudinal Displacement):** This represents the depth or length of the hallway. Because the prototype is moved manually in 250 mm increments, the X-coordinate represents the distance from the starting point of the first scan.

ii. Hardware Setup Instructions

- Place the mapping apparatus on the corner of a stable chair or rolling cart. The ToF sensor must be positioned at the very edge of the platform to ensure a clear, unobstructed field of view for the Y and Z scan.
- Ensure that the wires connecting the MCU to the ToF sensor are long enough to accommodate a full 360° rotation. This will prevent any obstruction of the sensor's view and allow the stepper motor to rotate freely, ensuring proper functionality.
- Connect the board to the PC. The system will undergo an initialization phase indicated by the onboard LEDs (see LED Status Logic below).

iii. Procedure

- Position the chair with the ToF scanner on the edge at the starting position ($X = 0$) and press PJ0 on the board. The stepper motor will rotate in a clockwise direction, and the sensor will rotate 360° to capture 32 points.
- After the scan is complete, the motor will automatically unwind in a counterclockwise direction to its original position to prevent cable tangling. Wait for the LED feedback to confirm the cycle is complete.
- Physically move the chair/cart forward along the hallway by exactly 250 mm.
- Press PJ0 again to capture the next slice. Continue this process until the desired number of scans for the hallway is achieved.
- Open and run the MATLAB script on the host PC. As soon as the command window prompts: "Press PJ1 on board to send data....," immediately press PJ1 on the MSP432 board.
- The command window will display the raw data being received. Once all packets are processed, the 3D map will automatically appear on the screen.

iv. LED Feedback

The onboard LEDs provide real-time feedback on the system's state. The indicators are as follows:

- **Initialization:** All LEDs Flash Once immediately after PLL_Init() and onboardLEDs_Init() confirming all LEDs are functional
- **ToF Boot Check:** D3(PF4) Flashes when sensor returns sensorState==1 which confirms when VL53L1X sensor has finished initial boot.
- **System Ready:** D2(PN0) Flashes. Indicates all ports are initialized, the sensor is ranging, and the system is waiting for the user to press PJ0.
- **CW Scan Phase:** D4(PF0) Solid ON throughout the entire 360° clockwise rotation
- **Measurement Capture:** D2(PN0) Flashes 32 times during a scan. Each flash indicates one successful distance measurement was captured.
- **CCW Unwind Phase:** D3(PF4) Solid ON. Turns ON after the scan is complete. Stays ON while the motor returns 4096 steps back to the home position.
- **UART Tx Start:** D1(PN1) Flashes Once. Occurs immediately after PJ1 is pressed. Signals that the UART transmission block to MATLAB has begun.
- **Critical error and Warning Indication:** All LEDs Flash when the following errors occur:
 - ToF Initialization Error (3 Flashes): If the ToF sensor fails to boot

(sensorState == 0), if SensorInit fails, or if StartRanging fails, all LEDs will flash 3 times.

- Memory Capacity Warning (2 Flashes): If you attempt to start a scan but scanNum has reached MAX_SCANS (25), all LEDs will flash 2 times.
- Empty Data Warning (3 Flashes): If you press PJ1 before performing any scans, all LEDs will flash 3 times.

v. **Expected Output**

To validate the accuracy of, a comparison is provided below between the physical campus location and the digital reconstruction. AC8874 Device and how to place it on the chair has also been attached.

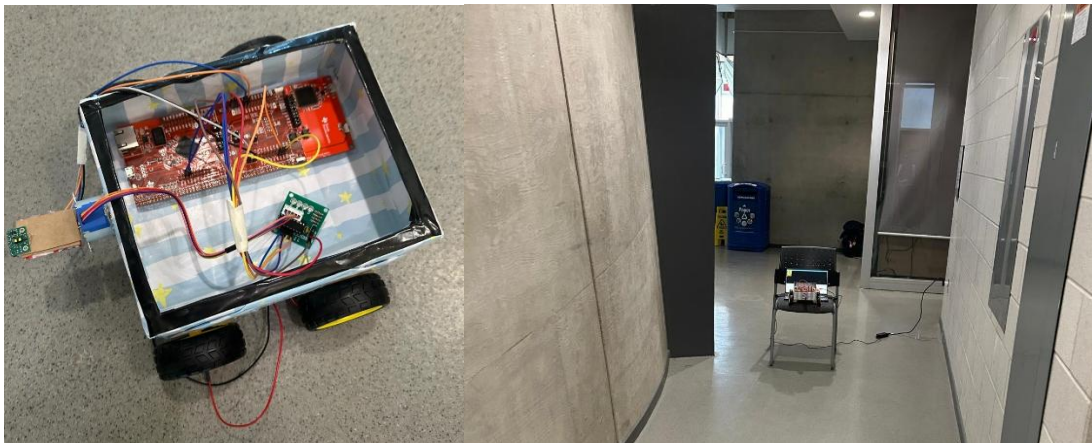


Figure 2 (left) AC8874 device (right) the device's placement on the chair when taking the scan



Figure 3: (left) adjacent hallway, (middle) hallway represented by E, (right) image from the floorplan

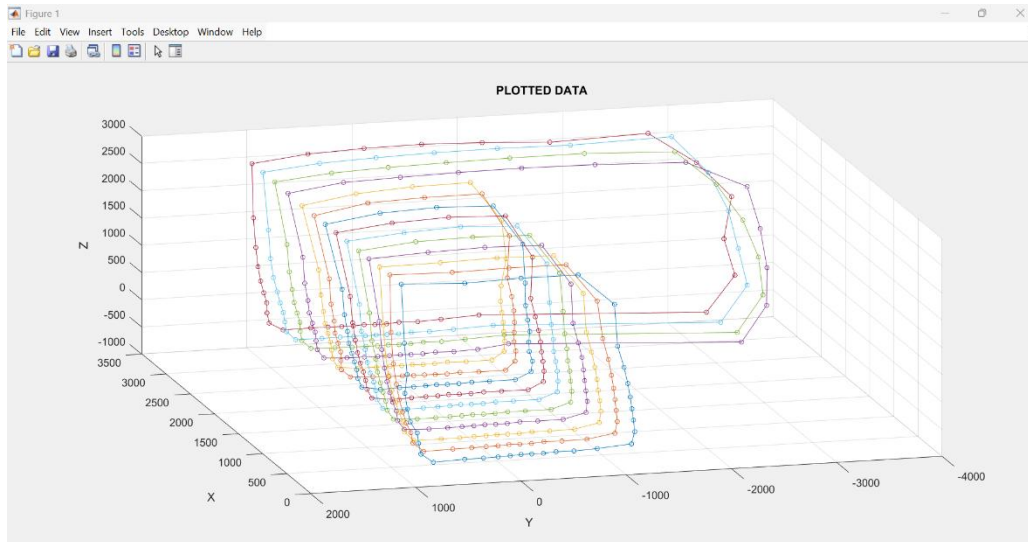


Figure 4: Front view of the scan

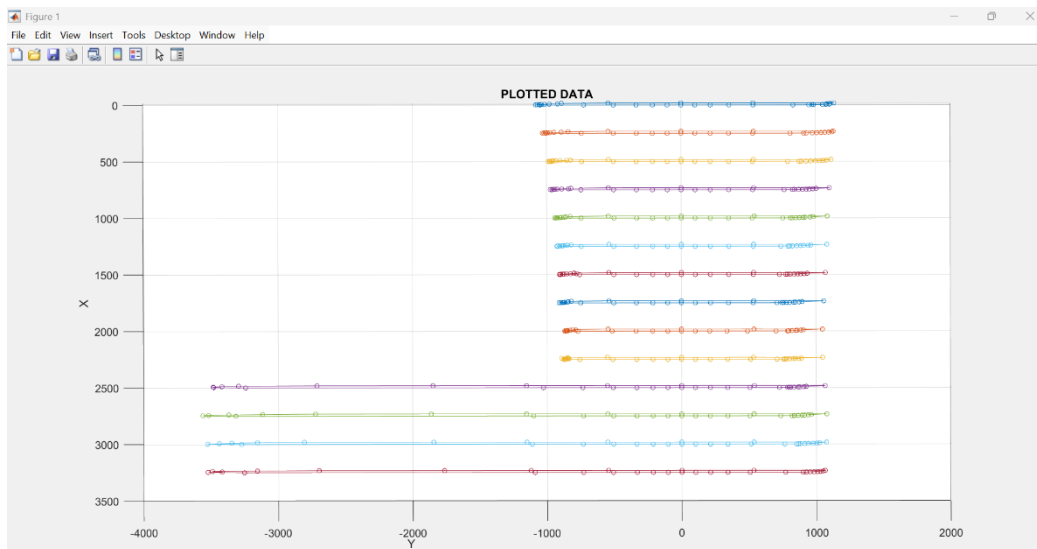


Figure 5: Top View of the Scan

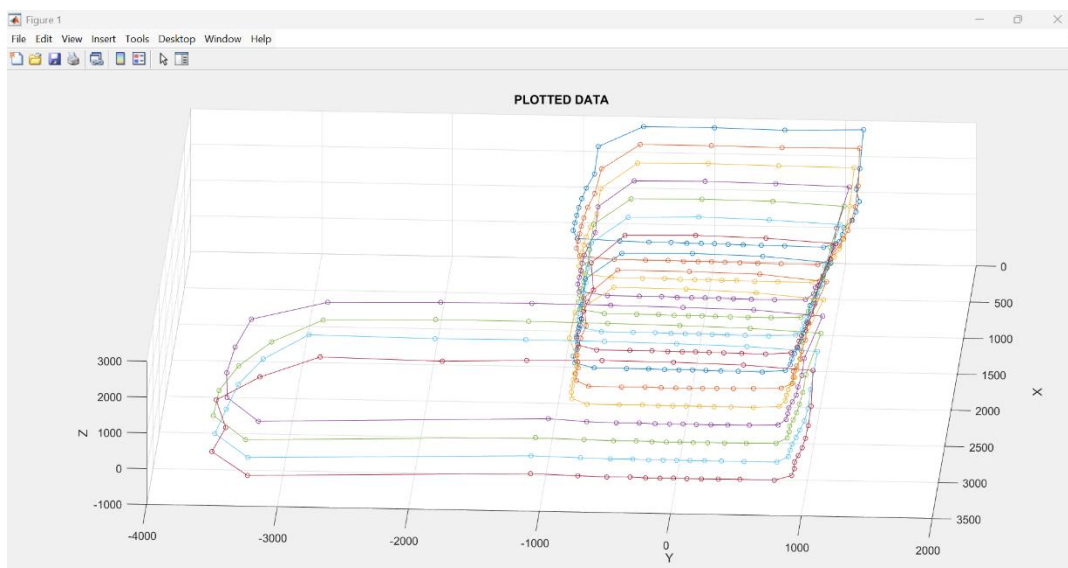


Figure 6: Back view of the scan

Limitations

The MSP432E401Y features an ARM Cortex-M4F core with a Floating Point Unit (FPU) designed mainly for simple math operations like addition and multiplication. However, it struggles with more complex functions like sine and cosine because it lacks built-in hardware instructions for these calculations. Instead, these functions are computed using software libraries that rely on power series approximations, which take much longer and can introduce delays. Additionally, the FPU typically handles 32-bit single-precision values, making trigonometric calculations susceptible to rounding errors, especially when converting from degrees to radians. This combination of challenges can create a processing bottleneck. If the microcontroller tries to calculate every XYZ coordinate in real-time while controlling the motor, the processing delays could disrupt the timing of the stepper motor, resulting in mechanical jitter and inaccuracies in spatial data.

The maximum standard serial communication rate implemented with the PC for this project is 128,000 bits per second (128 kbps). This limit was verified through the Windows Device Manager by accessing the properties of the XDS110 Class Application/User UART port. Under the "Port Settings" tab, the "Bits per second" drop-down menu lists 128,000 as the highest available standard selection for the PC's serial driver configuration. While the MSP432E401Y datasheet specifies that the internal UART modules are capable of much higher speeds, up to 7.5 Mbps in regular mode and 15 Mbps in high-speed mode, the practical communication speed for this application is governed by the standard options supported by the PC's operating system and the driver interface for the onboard USB-to-UART bridge.

The communication between the MSP432E401Y microcontroller and the VL53L1X ToF sensor is implemented using the I2C (Inter-Integrated Circuit) protocol. In this architecture, the microcontroller operates as the Leader (Master), initiating all data requests and controlling the clock line, while the ToF sensor acts as the Follower (Slave). The system is currently configured for a communication rate of 100 kbps (Standard Mode). This speed was selected to ensure high signal integrity over the physical wire connections between the sensor and the board. However, it is important to note the hardware's theoretical capabilities: while the MSP432E401Y supports high-speed I2C communication up to 3.33 Mbps, and the VL53L1X sensor supports Fast Mode up to 400 kbps, the system's operational speed is intentionally capped at 100 kbps. This rate is verified within the `I2C_Init()` function, where the Master Timer Period Register (`I2C0_MTPR_R`) is calculated based on the 22MHz bus clock to achieve the designated 100kHz frequency.

Reviewing the entire system architecture, the primary limitation on operational speed is the mechanical latency of the stepper motor, followed by secondary delays from the ToF sensing window and the UART serial transmission. To ensure the motor rotates accurately without losing torque or slipping, a mandatory 2 ms delay must be implemented between each step in the 8-step half-step sequence. Because this step sequence requires 4,096 half-steps for a single 360-degree rotation, the motor logic creates a hard mechanical time floor of approximately 8.192 seconds per scan. This is calculated using the formula $T_{\text{motor}} = 4096 \text{ steps} \times (0.002 \text{ sec/step}) = 8.192 \text{ seconds}$, which accounts for the vast majority of the system's active cycle. In addition to these timing constraints, the system is influenced by quantization error, which defines the precision limits of the digital measurements. Since the VL53L1X ToF module maps its 4000 mm range to a 16-bit integer (2^{16} or 65,536 levels), the resolution is calculated by the formula $\text{Quantization Error} = 4000\text{mm}/65536$, resulting in a precision limit of approximately 0.061 mm.

The identification of these bottlenecks was verified through differential testing, a process where specific blocks of code were systematically commented out to measure the time savings per rotation. Mechanical verification was performed by attempting to reduce the step delay below the 2 ms threshold, which resulted in the motor vibrating in place rather than rotating, confirming the hardware's physical speed limit. Sensing verification involved running the project without the ToF sensor by commenting out all API and I2C calls, which saved less than 200 ms per rotation. Finally, transmission verification was conducted by disabling the UART data transfer to the PC, which saved less than 100 ms per rotation at the standard rate of 128 kbps. This comprehensive analysis demonstrates that the mechanical movements of the motor are nearly 40 times slower than the electronic sensing and transmission combined. Consequently, while the 16-bit sensor architecture provides sub-millimetre data precision, the stepper motor's physical timing requirements serve as the defining bottleneck for the entire system's speed.

Circuit Schematic

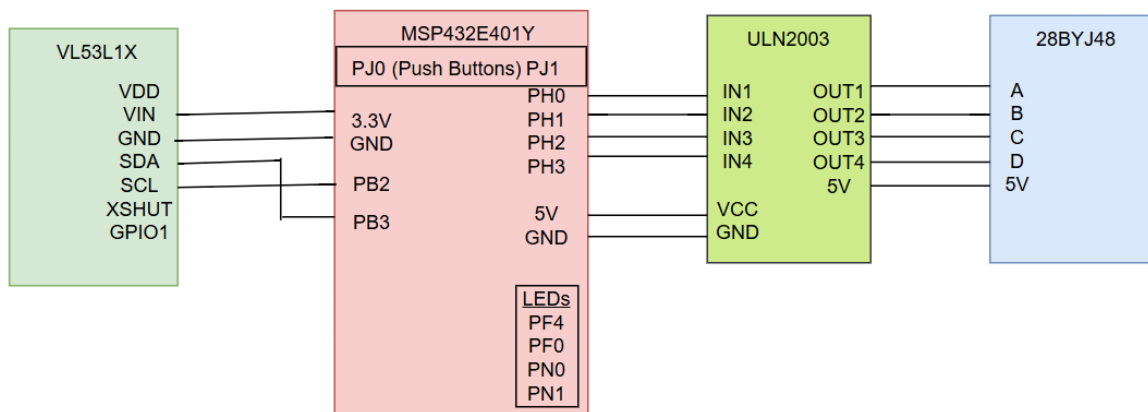


Figure 7: Circuit Schematic

Programming Logic Flowchart(s)

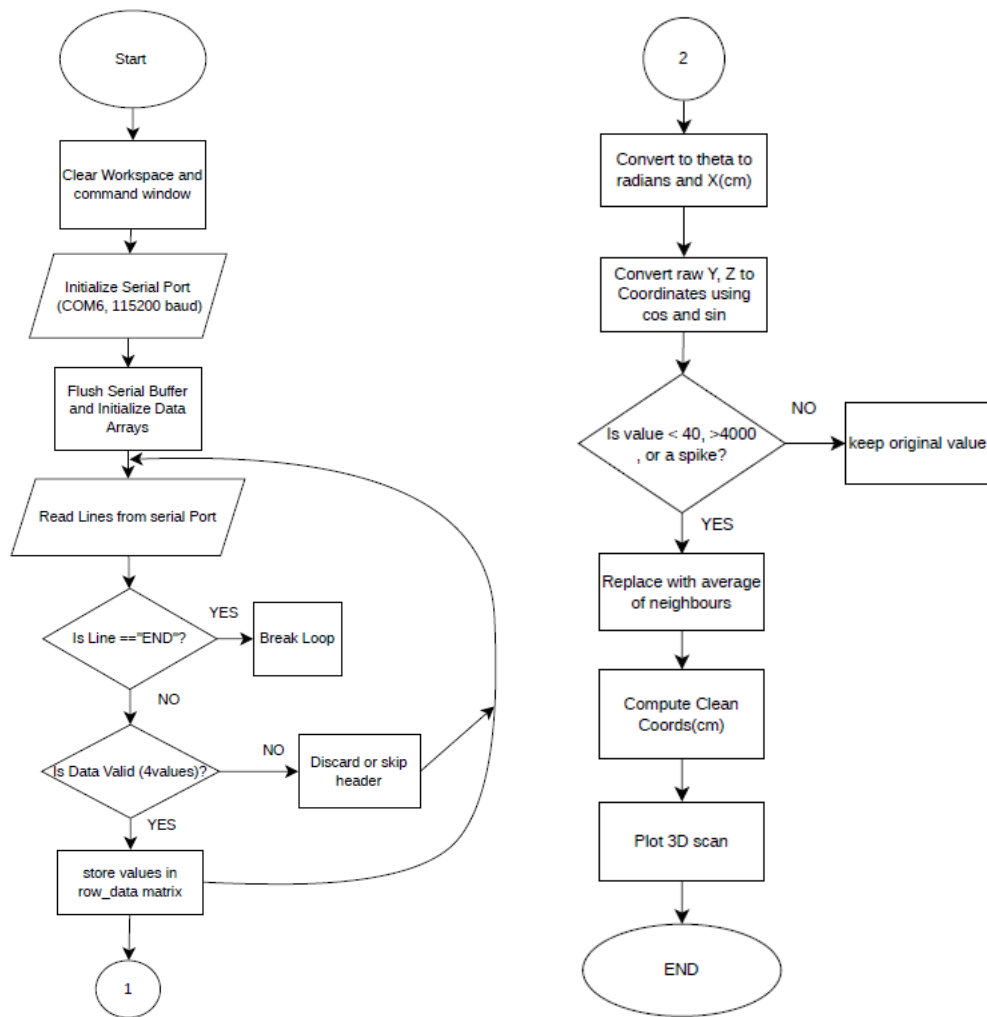
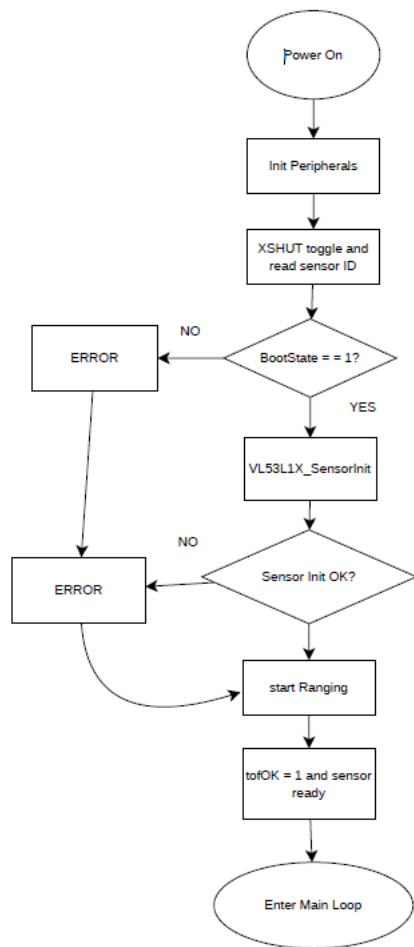


Figure 8: MATLAB Flowchart



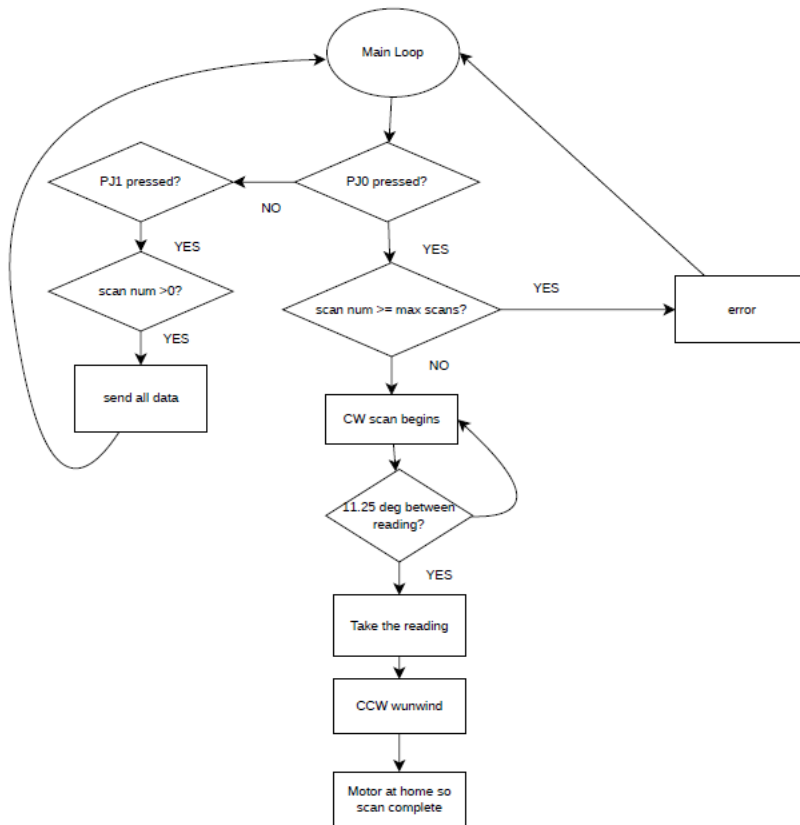


Figure 9: Kiel program flowchart

References

- [1] “VL53L1X,” Pololu, <https://www.pololu.com/file/0J1506/vl53l1x.pdf> [accessed Apr. 7, 2026].
- [2] “MSP432E4 SimpleLink™ Microcontrollers Technical Reference Manual,” Texas Instruments, <https://www.ti.com/lit/ug/slau723a/slau723a.pdf> [accessed Apr. 7, 2026].